

Assignment 7 – SQL & PySpark Interview Preparation

These questions were shared for interview preparation. Each problem is explained with the reasoning first, followed by SQL and PySpark approaches where applicable.

Question 1 — Delivery Partner: Total Rides, Rides Won and Rides Lost

Given Delivery_Partner with Brand_1, Brand_2, Brand_3 and Winner, find Total Rides, Rides Won and Rides Lost for every brand.

Brand_1	Brand_2	Brand_3	Winner
A	B	C	B
B	C	E	E
C	A	D	D
D	E	A	A
F	B	C	F

Understand the problem

Each row is one ride. Three brands participated and one brand won. Therefore Brand_1, Brand_2 and Brand_3 represent participation; Winner identifies the winner.

Brand_1 + Brand_2 + Brand_3
↓
participating brands
↓
COUNT(*) → total rides
brand = winner → rides won
total rides - rides won → rides lost

SQL

```
WITH all_brands AS (  
  SELECT brand_1 AS brand, winner FROM delivery_partner  
  UNION ALL  
  SELECT brand_2 AS brand, winner FROM delivery_partner  
  UNION ALL  
  SELECT brand_3 AS brand, winner FROM delivery_partner  
)  
result AS (  
  SELECT  
    brand,  
    COUNT(*) AS no_of_rides,  
    SUM(CASE WHEN brand = winner THEN 1 ELSE 0 END) AS rides_won  
  FROM all_brands  
  GROUP BY brand  
)  
SELECT  
  brand,  
  no_of_rides,  
  rides_won,  
  no_of_rides - rides_won AS rides_lost  
FROM result  
ORDER BY brand;
```

Expected Result

Brand	No of Rides	Rides Won	Rides Lost
A	3	1	2
B	3	1	2
C	4	0	4
D	2	1	1

Brand	No of Rides	Rides Won	Rides Lost
E	2	1	1
F	1	1	0

PySpark

```
from pyspark.sql.functions import *

brands_df = delivery_df.selectExpr(
    "stack(3, Brand_1, Winner, Brand_2, Winner, Brand_3, Winner) as (brand, winner)"
)

result_df = (
    brands_df
    .groupBy("brand")
    .agg(
        count("*").alias("no_of_rides"),
        sum(when(col("brand") == col("winner"), 1).otherwise(0))
        .alias("rides_won")
    )
    .withColumn("rides_lost", col("no_of_rides") - col("rides_won"))
    .orderBy("brand")
)

result_df.show()
```

Interview approach: Normalize/unpivot the three Brand columns into rows, group by brand, count participation, count wins, then calculate losses.

Question 2 — Print All Consecutive 'a' Characters in a String

Input: taabaaacdaaaadfdag. Required output: ['aa', 'aaa', 'aaaa', 'a'].

```
t aa b aaa c d aaaa d f d a g
  ↑   ↑   ↑       ↑
  aa  aaa  aaaa   a
```

Python

```
import re

text = "taabaaacdaaaadfdag"
result = re.findall(r'a+', text)
print(result)
```

a+ means one or more consecutive a characters.

PySpark — Without UDF and Without Regex

The approach is: split the string into characters, give each character a position, use LAG() to inspect the previous character, identify the beginning of every a-run, assign a group ID, count each group and reconstruct the a sequence.

```
String
↓
characters + positions
↓
LAG(previous character)
↓
find start of each 'a' run
↓
cumulative SUM → group_id
↓
count each group
↓
repeat('a', count)
```

Complete PySpark Solution

```
from pyspark.sql.functions import *
from pyspark.sql.window import Window

data = [("taabaaacdaaaadfdag",)]
df = spark.createDataFrame(data, ["text"])

window = Window.orderBy("pos")

result = (
```

```

df
.withColumn("pos", explode(sequence(lit(1), length("text"))))
.withColumn("char", substring("text", col("pos"), 1))
.withColumn("previous_char", lag("char").over(window))
.withColumn(
  "new_group",
  when(
    (col("char") == "a") &
    ((col("previous_char") != "a") |
     col("previous_char").isNull()),
    1
  ).otherwise(0)
)
.withColumn(
  "group_id",
  sum("new_group").over(
    window.rowsBetween(
      Window.unboundedPreceding,
      Window.currentRow
    )
  )
)
.filter(col("char") == "a")
.groupBy("group_id")
.agg(count("*").alias("a_count"))
.withColumn("consecutive_a", repeat(lit("a"), col("a_count")))
.orderBy("group_id")
.select("consecutive_a")
)

result.show()

group_id | a_count
-----+-----
1        | 2      → aa
2        | 3      → aaa
3        | 4      → aaaa
4        | 1      → a

```

Question 3 — Employees Who Joined Before Their Managers

Given emp_id, emp_name, manager_id and join_date, return employees whose join_date is earlier than their manager's join_date.

emp_id	emp_name	manager_id	join_date
1	Alice	3	5/10/2018
2	Bob	3	3/15/2019
3	Charlie	4	1/1/2020
4	David	NULL	8/20/2017
5	Eva	3	11/5/2017

The manager is also an employee in the same table, so this is a **self-join**.

```

employees e → employee
employees m → manager

e.manager_id = m.emp_id
e.join_date < m.join_date

```

SQL

```

SELECT
  e.emp_name,
  m.emp_name AS manager_name
FROM employees e
JOIN employees m
  ON e.manager_id = m.emp_id
WHERE e.join_date < m.join_date;

```

Result based on the input shown

emp_name	manager_name
Alice	Charlie
Bob	Charlie
Eva	Charlie

Note: The image's displayed expected output omits Bob. Based on the input shown, Bob joined on 3/15/2019 and Charlie on 1/1/2020, so Bob also satisfies the stated condition.

PySpark

```
from pyspark.sql.functions import col

e = employees.alias("e")
m = employees.alias("m")

result = (
    e.join(m, col("e.manager_id") == col("m.emp_id"), "inner")
    .filter(col("e.join_date") < col("m.join_date"))
    .select(
        col("e.emp_name").alias("emp_name"),
        col("m.emp_name").alias("manager_name")
    )
)

result.show()
```

Interview pattern: Employee → Manager in the same table = think SELF JOIN.

Question 4 — Elements Appearing Three Times Consecutively

Return distinct elements that appear at least three times consecutively.

A
B
B
A
A
A
B
C
D
C
C
C
C
C

The qualifying runs are A A A and C C C C, so the distinct result is A and C.

SQL

A sequence problem needs a defined order. The image does not show an ordering column, so the query assumes an id/order column.

```
WITH x AS (
    SELECT
        id,
        element,
        LAG(element, 1) OVER (ORDER BY id) AS prev_1,
        LAG(element, 2) OVER (ORDER BY id) AS prev_2
    FROM elements
)
SELECT DISTINCT element
FROM x
WHERE element = prev_1
AND element = prev_2;
```

When the current value equals the previous two values, there are at least three consecutive occurrences. DISTINCT prevents a longer run from producing duplicate output.

PySpark

```
from pyspark.sql.window import Window
from pyspark.sql.functions import *

window = Window.orderBy("id")
```

```

result = (
    df
    .withColumn("prev_1", lag("element", 1).over(window))
    .withColumn("prev_2", lag("element", 2).over(window))
    .filter(
        (col("element") == col("prev_1")) &
        (col("element") == col("prev_2"))
    )
    .select("element")
    .distinct()
)

```

result.show()

Need previous rows

↓
LAG()

↓
Compare current with previous 2

↓
Three equal values → qualifies

↓
DISTINCT

Interview pattern: Consecutive-value problems need an ordering column plus a window function such as LAG().

Question 5 — Categorize Products Based on Price

Create price_category using: price < 100 → Low; 100 <= price < 500 → Medium; price >= 500 → High.

product	price
Laptop	800
Mouse	25
Keyboard	150
Monitor	300

SQL

```

SELECT
    product,
    price,
    CASE
        WHEN price < 100 THEN 'Low'
        WHEN price >= 100 AND price < 500 THEN 'Medium'
        WHEN price >= 500 THEN 'High'
        ELSE 'Unknown'
    END AS price_category
FROM products;

```

Expected Output

product	price	price_category
Laptop	800	High
Mouse	25	Low
Keyboard	150	Medium
Monitor	300	Medium

PySpark

```

from pyspark.sql.functions import *

result = df.withColumn(
    "price_category",
    when(col("price") < 100, "Low")
    .when(
        (col("price") >= 100) &
        (col("price") < 500),
        "Medium"
    )
)

```

```

        .when(col("price") >= 500, "High")
        .otherwise("Unknown")
    )

    result.show()

```

This is a built-in Spark transformation using `when()` and `otherwise()`; a UDF is not needed because the logic can be expressed with native Spark column expressions.

Assignment 7 — Interview Pattern Summary

Question	Main Concept
Q1 – Delivery Partner	Unpivot/stack + GROUP BY + conditional aggregation
Q2 – Consecutive 'a'	Character processing + LAG() + Window
Q3 – Employee/Manager	Self Join
Q4 – Three consecutive elements	LAG() + Window
Q5 – Price Category	CASE WHEN / when() + otherwise()

Patterns to Remember

Multiple same-type columns

↓

UNPIVOT / STACK

↓

GROUP BY + AGGREGATE

Need previous row → LAG()

Need next row → LEAD()

Employee → Manager → SELF JOIN

Multiple conditions → CASE WHEN / when().otherwise()

Consecutive values → ordering column + Window + LAG()

Interview lesson: Before writing code, identify the data pattern. Then choose the SQL/PySpark operation that matches it.